Lecture 34 — Square-Root Decomposition & Mo's Algorithm

> Year Curriculum · Prerequisites: Week 3 (complexity), Week 7 (prefix sums), Weeks 23–25 (segment trees, Fenwick, lazy propagation) · Estimated time: 2.5 hours

1. Overview

Segment trees answer most range-query problems in O(log n), but they require that the information of two halves can be **merged** quickly, and lazy propagation can be painful to get right. **Square-root decomposition** trades the logarithm for a √n factor in exchange for enormous flexibility: split the array (or the queries, or the parameter space) into about √n pieces, handle "small" parts by brute force and "large" parts by precomputed summaries.

This lecture covers:

1. **Block decomposition** of an array: point and range updates, sums, and non-mergeable queries.
2. **In-block jump pointers** (CF 13E Holes).
3. **Thresholding** (small-parameter vs large-parameter cases).
4. **Sqrt decomposition on queries** (periodic rebuilding).
5. **Mo's algorithm** for offline range queries.

With $n, q \le 2 \cdot 10^5$, an $O((n + q)\sqrt{n})$ algorithm performs about 10^8 simple operations, which typically runs in under a second in C++ if the inner loops are tight.

2. Block decomposition

2.1 The basic structure

Choose a block size B and put index i in block $\lfloor i / B \rfloor$. There are $\lceil n / B \rceil$ blocks; the last one may be shorter. For each block store a summary — for sums, the block's total.

A query on [l, r]:

- walks element by element through the **partial** block at the left end, until l is at a block boundary;
- jumps over **full** blocks using their summaries;
- walks element by element through the partial block at the right end.

That is at most 2B element steps plus n/B block steps, O(B + n/B), minimized by **B ≈ √n**, giving O(√n). A point update changes one element and one block summary in O(1).

Worked example, n = 10, B = 3: a = [3, 1, 4 | 1, 5, 9 | 2, 6, 5 | 3], block sums [8, 15, 13, 3]. Query [1, 7] = a[1] + a[2] + block 1 + a[6] + a[7] = 1 + 4 + 15 + 2 + 6 = 28.

2.2 Range updates with lazy tags

For "add x to every element in [l, r]" and range-sum queries, store per block `sum[b]` (the true sum of the block, including pending additions) and `add[b]` (a value to be added to every element of the block, not yet written into `a`).

- Update: for each full block, `add[b] += x` and `sum[b] += x · len(b)`; for each element in a partial block, `a[i] += x` and `sum[b] += x`.
- Query: full blocks contribute `sum[b]`; elements of partial blocks contribute `a[i] + add[b]`.

Unlike a segment tree, we never need to "push" tags down, because a tag lives on exactly one level. Both operations are O(√n). Variants (assign instead of add, min instead of sum) work the same way, but an assignment tag must override existing element values, so when a partial update hits a block with an active assignment tag, first write the tag into all its elements (O(B)) and clear it.

2.3 Non-mergeable summaries

The real strength of blocks is that the summary can be anything we can rebuild in O(B) or O(B log B). Example: point updates and queries "how many elements of a[l..r] are ≤ x". Keep a sorted copy of each block. A query binary-searches each full block (O(log B) each) and checks partial elements directly: O(√n log n). An update replaces one value in one sorted block (O(B) by shifting). Choosing B ≈ √(n log n) balances the two terms of the query. With a segment tree the equivalent structure is a merge-sort tree, which does not support updates easily.

2.4 In-block jump pointers: CF 13E Holes

n holes in a row, hole i has power a[i]: a ball at i jumps to i + a[i], until it leaves the row. Queries: change a[i], or throw a ball into hole i and report the last hole it visits and the number of jumps.

For every i, precompute information **restricted to i's block**:

- `nxt[i]` — the first position reached by the ball from i that lies **outside** i's block (possibly > n − 1, meaning it left the row);
- `cnt[i]` — the number of jumps needed to get there;
- `last[i]` — the last hole visited inside the block.

Compute them for a block from right to left: let j = i + a[i]; if j is outside the block, then nxt[i] = j, cnt[i] = 1, last[i] = i; otherwise nxt[i] = nxt[j], cnt[i] = cnt[j] + 1, last[i] = last[j].

A throw follows `nxt` pointers, one block at a time (positions only increase, so at most n/B + 1 hops), summing `cnt` and remembering `last`: O(n/B). An update only invalidates i's own block (pointers from other blocks never look inside it except through `nxt`, which is computed fresh when needed): recompute that block in O(B). With B = √n both are O(√n).

3. Thresholds: small vs large

Many problems have a parameter that makes them easy when it is large and makes precomputation cheap when it is small.

CF 797E Array Queries. Array a with 1 ≤ a[i] ≤ n. Query (p, k): repeatedly set p ← p + a[p] + k while p ≤ n; output the number of steps.

- If k > √n, each step advances more than √n positions, so simulating takes at most √n steps.
- If k ≤ √n, there are only √n different values of k. For each such k precompute `dp[k][p]` = answer for start p, from right to left: `dp[k][p] = 1 + dp[k][p + a[p] + k]` (0 if p > n).

Total O(n√n + q√n) time and O(n√n) memory (use `int`, and consider a slightly smaller threshold if memory is tight).

The same idea appears on graphs: a vertex is **heavy** if its degree exceeds √m; there are at most 2√m heavy vertices (degrees sum to 2m). Light vertices can afford to scan their neighbours on each operation, while heavy vertices keep precomputed aggregates. Triangle counting in O(m√m) is the classic example.

4. Sqrt decomposition on queries

Sometimes updates are hard to integrate into a structure, but rebuilding the structure from scratch is O(n). Then:

- maintain a **buffer** of the updates since the last rebuild;
- answer each query from the rebuilt structure, then correct it by scanning the buffer (O(|buffer|) per query);
- when the buffer reaches size K, rebuild (O(n)) and clear it.

Total cost O(q·K + (q/K)·n), minimized at K ≈ √n. Example: "mark a vertex red / query the distance from v to the nearest red vertex" on a tree: rebuild with multi-source BFS every K updates; for a query, take the BFS distance and compare with the (LCA-based) distances to the ≤ K buffered red vertices.

5. Mo's algorithm

5.1 The idea

Suppose all q range queries are known in advance (offline), there are no updates, and we can maintain the answer for a "current window" [L, R] when it grows or shrinks by one element — `add(i)` and `remove(i)` in O(1). Answering the queries in input order might move the window O(n) per query, for O(nq) total. Mo's algorithm **reorders the queries** so that the total movement is small.

5.2 Ordering and complexity

Sort the queries by the pair ($\lfloor l / B \rfloor$, r). Analyze the pointer movement:

- **Right pointer.** Among queries whose l lies in the same block, r is non-decreasing, so R moves at most n in total per block. With n/B blocks: $O(n^2 / B)$.
- **Left pointer.** Between consecutive queries in the same block, l moves at most B; across a block boundary at most 2B. Total $O(q \cdot B)$.

The total $O(n^2/B + qB)$ is minimized at $\mathbf{B = n / \sqrt{q}}$, giving $\mathbf{O(n\sqrt{q})}$ pointer moves, i.e. $O(n\sqrt{n})$ when $q \approx n$. Each move costs one `add` or `remove` call.

5.3 Implementation details

- Keep the window as [L, R] with the empty window $L = 0$, $R = -1$.
- Move pointers so the window never becomes invalid: first **expand** (R up, L down), then **shrink** (R down, L up). Shrinking first can call `remove` on elements that are not in the window, which corrupts counters.
- Store each answer at its query's original index.
- **Odd-even optimization:** in odd-numbered blocks sort r in decreasing order. R then sweeps back and forth instead of returning to the left after each block, roughly halving the running time.
- **Coordinate compression:** if values are up to 10^9 and you count occurrences, compress values to $0..n - 1$ first.
- An even better constant comes from sorting queries by their index along a **Hilbert curve** over the (l, r) plane.

5.4 Example: distinct values (CSES 1734, SPOJ DQUERY)

Maintain `cnt[v]` = occurrences of value v in the window and `cur` = number of v with cnt[v] > 0. `add` increments cnt and increments cur when cnt goes 0 → 1; `remove` decrements and decrements cur when cnt goes 1 → 0.

5.5 Example: Powerful Array (CF 86D)

The power of a subarray is $\Sigma_s K_s^2 \cdot s$ where K_s is the number of occurrences of s. When K_s increases by one the answer changes by $((K_s+1)^2 - K_s^2)\cdot s = (2K_s+1)\cdot s$, and symmetrically for removal. The answer can reach about 10^{18} ($n = 2\cdot10^5$, values up to 10^6): use `long long`. This illustrates the main appeal of Mo's algorithm: the aggregate is awkward to merge (no segment tree), but its **delta** under adding one element is trivial.

5.6 Limitations and extensions

Mo's algorithm is offline and requires cheap add/remove. If removal is hard (e.g., maintaining a maximum), use **Mo's with rollback**, which only adds and then restores a snapshot. With point updates, add a time dimension (**Mo's with updates**, O(n^{5/3}), World Finals lecture 5). Path queries on trees can be turned into range queries on an Euler tour (**Mo on trees**).

6. Reference implementations

6.1 Range add / range sum with lazy blocks

Input: n q, the array, then queries `1 l r x` (add x to a[l..r]) or `2 l r` (print the sum), 1-based.

```cpp
#include <bits/stdc++.h>
using namespace std;

struct SqrtArray {
    int n, B;
    vector<long long> a, sum, add;
    SqrtArray(const vector<long long>& v) : n(v.size()), a(v) {
        B = max(1, (int)sqrt(n));
        int nb = (n + B - 1) / B;
        sum.assign(nb, 0);
        add.assign(nb, 0);
        for (int i = 0; i < n; i++) sum[i / B] += a[i];
    }
    int blockLen(int b) const { return min(n, (b + 1) * B) - b * B; }
    void rangeAdd(int l, int r, long long x) {
        while (l <= r && l % B != 0) { a[l] += x; sum[l / B] += x; l++; }
        while (l + B - 1 <= r) { add[l / B] += x; sum[l / B] += x * B; l += B; }
        while (l <= r) { a[l] += x; sum[l / B] += x; l++; }
    }
    long long rangeSum(int l, int r) const {
        long long s = 0;
        while (l <= r && l % B != 0) { s += a[l] + add[l / B]; l++; }
        while (l + B - 1 <= r) { s += sum[l / B]; l += B; }
        while (l <= r) { s += a[l] + add[l / B]; l++; }
        return s;
    }
```

```
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<long long> v(n);
    for (auto& x : v) cin >> x;
    SqrtArray st(v);
    while (q--) {
        int type, l, r;
        cin >> type >> l >> r;
        --l; --r;
        if (type == 1) {
            long long x;
            cin >> x;
            st.rangeAdd(l, r, x);
        } else {
            cout << st.rangeSum(l, r) << "\n";
        }
    }
}
```

The middle loops only ever touch **complete** blocks (l is a multiple of B and l + B − 1 ≤ r), so `x * B` is the right amount even though the last block may be shorter; `blockLen` is there for variants (such as assignment tags) that need it.

6.2 CF 13E Holes

```
#include <bits/stdc++.h>
using namespace std;

int n, B;
vector<int> a, nxt, cnt, lst;

void rebuildBlock(int b) {
    int lo = b * B, hi = min(n, lo + B);  // block covers [lo, hi)
    for (int i = hi - 1; i >= lo; i--) {
        int j = i + a[i];
        if (j >= hi) { nxt[i] = j; cnt[i] = 1; lst[i] = i; }
        else { nxt[i] = nxt[j]; cnt[i] = cnt[j] + 1; lst[i] = lst[j]; }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> n >> m;
    a.resize(n); nxt.resize(n); cnt.resize(n); lst.resize(n);
    for (auto& x : a) cin >> x;
```

```
    B = max(1, (int)sqrt(n));
    for (int b = 0; b * B < n; b++) rebuildBlock(b);
    while (m--) {
        int type;
        cin >> type;
        if (type == 0) {
            int i, v;
            cin >> i >> v;
            a[--i] = v;
            rebuildBlock(i / B);
        } else {
            int i;
            cin >> i;
            --i;
            int jumps = 0, last = i;
            while (i < n) {
                jumps += cnt[i];
                last = lst[i];
                i = nxt[i];
            }
            cout << last + 1 << " " << jumps << "\n";
        }
    }
}
```

Sample (from the problem): `8 5 / 1 1 1 1 1 2 8 2 / 1 1 / 0 1 3 / 1 1 / 0 3 4 / 1 2` → `8 7`, `8 5`, `7 3`.

6.3 Mo's algorithm — CSES 1734 Distinct Values Queries

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<int> a(n);
    for (auto& x : a) cin >> x;
    vector<int> vals(a);  // coordinate compression
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (auto& x : a) x = lower_bound(vals.begin(), vals.end(), x) - vals.begin();

    struct Q { int l, r, id; };
    vector<Q> qs(q);
    for (int i = 0; i < q; i++) {
        cin >> qs[i].l >> qs[i].r;
        qs[i].l--; qs[i].r--;
        qs[i].id = i;
    }
```

```
    int B = max(1, (int)(n / sqrt((double)q)));
    sort(qs.begin(), qs.end(), [&](const Q& x, const Q& y) {
        if (x.l / B != y.l / B) return x.l / B < y.l / B;
        return (x.l / B & 1) ? x.r > y.r : x.r < y.r;
    });

    vector<int> cnt(vals.size(), 0), ans(q);
    int cur = 0, L = 0, R = -1;
    auto add = [&](int i) { if (cnt[a[i]]++ == 0) cur++; };
    auto remove = [&](int i) { if (--cnt[a[i]] == 0) cur--; };
    for (const Q& qu : qs) {
        while (R < qu.r) add(++R);
        while (L > qu.l) add(--L);
        while (R > qu.r) remove(R--);
        while (L < qu.l) remove(L++);
        ans[qu.id] = cur;
    }
    for (int x : ans) cout << x << "\n";
}
```

Sample: `5 3 / 3 2 3 1 2 / 1 3 / 2 4 / 1 5` → 2, 3, 3.

7. Common mistakes

1. **B = 0.** `sqrt(0)` or `n / sqrt(q)` with tiny n gives 0 and a division by zero. Always `max(1, …)`.
2. **The last block is shorter.** Any code that assumes every block has exactly B elements (e.g., `sum[b] += x * B` on a block that is not full) is wrong unless it only applies to full blocks, as in 6.1.
3. **Forgetting the lazy tag** when reading single elements in partial blocks.
4. **Stale block data after an update.** In Holes-style structures, rebuild the whole affected block, right to left.
5. **Mo's pointer order.** Expand before shrinking. Shrinking first can remove an element that was never added.
6. **Answers in sorted order.** Store by original query index and print in input order.
7. **Uncompressed values** as array indices (values up to 10^9), or a counter array sized by n when values go up to 10^6 (CF 86D).
8. **Overflow** of aggregate answers: use `long long`.
9. **Slow inner loops.** Mo's performs ~10^8 add/remove calls: avoid `map`, `set`, or `unordered_map` inside them; use plain arrays.

8. Practice problems

- CSES 1734 — Distinct Values Queries (https://cses.fi/problemset/task/1734)
- SPOJ DQUERY — D-query (https://www.spoj.com/problems/DQUERY/)
- Codeforces 86D — Powerful array (https://codeforces.com/problemset/problem/86/D)

- Codeforces 220B — Little Elephant and Array (https://codeforces.com/problemset/problem/220/B)
- Codeforces 617E — XOR and Favorite Number (https://codeforces.com/problemset/problem/617/E)
- Codeforces 13E — Holes (https://codeforces.com/problemset/problem/13/E)
- Codeforces 797E — Array Queries (https://codeforces.com/problemset/problem/797/E)
- CSES 1648 — Dynamic Range Sum Queries (https://cses.fi/problemset/task/1648) — solve with blocks
- CSES 1651 — Range Update Queries (https://cses.fi/problemset/task/1651) — solve with lazy blocks
- Codeforces 455D — Serega and Fun (https://codeforces.com/problemset/problem/455/D) — stretch: blocks of deques

9. Further reading

- cp-algorithms: "Sqrt Decomposition" including Mo's algorithm (https://cp-algorithms.com/data_structures/sqrt_decomposition.html)
- Antti Laaksonen, *Competitive Programmer's Handbook*, chapter 27 (square root algorithms)
- Codeforces blog "An alternative sorting order for Mo's algorithm" (Hilbert order), blog entry 61203
- Codeforces blog "Mo's algorithm with updates" and "Mo's on trees" — preview of World Finals lecture 5